

Hermoso — QA Test Flow Guide

Web + Mobile · Prepared for QA testing with seeded/test accounts

This document walks QA through exercising the Hermoso platform end-to-end on both the **web app** and the **Android app**, using test accounts. Pair this with the Postman collection (docs/Hermoso.postman_collection.json) for direct API testing.

1. Environments

1.1 Live (Vercel) — primary environment for this QA pass

Component	URL
Backend API	https://hermoso-rx6j.vercel.app/ (/api/...)
Web app	https://hermoso-seven.vercel.app/
Mobile app	Latest APK (provided separately) — install directly on a device/emulator, no build step needed

The live database was reset for this QA pass (2026-08-10). It now contains only a fresh super admin account (Section 2) and the platform's category taxonomy — no salons, owners, staff, customers, bookings, or reviews. This is intentional: use the new **Salon Owners** (3.4, 7.1) and **Admin Access** (3.4, 7.2) flows to create your own fresh owner/admin test accounts as part of this pass, rather than relying on old seeded data. Do **not** run `npm run seed` (1.2) against this database — that would repopulate it with 100 mock salons/owners and defeat the point of the reset.

Also confirm the provided APK is built against `https://hermoso-rx6j.vercel.app/` — if a QA build accidentally points at a local/staging API, mobile requests will fail silently or time out, which is worth ruling out first if nothing loads.

1.2 Local dev setup (optional, for isolated/offline testing)

Component	Command	Default URL
API server	<code>cd server && npm run dev</code>	http://localhost:5000/api
Web app	<code>cd client && npm run dev</code>	Vite dev server (typically http://localhost:5173)
Mobile app	Open <code>android/</code> in Android Studio, run on emulator/device	Use <code>10.0.2.2:5000</code> for the emulator, or your LAN IP for a physical device
Seed test data	<code>cd server && npm run seed</code>	Wipes and repopulates the local database only

The web client's API base URL is set via `VITE_API_URL` (defaults to `http://localhost:5000/api`). Confirm the mobile build points at the same server as the web app before cross-platform testing locally.

2. Test Users

2.1 Live environment (current, post-reset)

Role	Email	Password	Notes
Super Admin	<code>admin@hermoso.app</code>	<code>Hermoso@wAppDJhF4oQa1</code>	The only account that exists right now. Full platform access, including managing other admin accounts. Rotate this password once QA wraps up.

Everything else — salon owners, staff, customers, salons, bookings — is **empty on the live DB by design** (see Section 1.1). Use this super admin login to:

- Create salon owner test accounts via **Admin** → **Salon Owners** (3.4, 7.1).
- Create additional admin test accounts via **Admin** → **Settings** → **Admin Access** (3.4, 7.2).
- Register fresh customer accounts yourself via `/register` (3.1) — customers aren't admin-creatable, they self-register.

Staff accounts still have no dedicated creation UI (see Section 6) — create them via the owner's Staff page (`/owner/staff`) once you have an owner account, or via the Postman collection.

2.2 Local dev seed data (from `server/scripts/seed.ts`, Section 1.2 only)

Running `npm run seed` in `server/` populates a **local** database with 100 mock salons/owners/staff/customers for offline testing. All seeded accounts are pre-verified, so they can log in immediately — no OTP step needed. **Do not run this against the live database** (see Section 1.1).

Role	Email pattern	Password	Notes
Super Admin	<code>admin@hermoso.app</code>	<code>Admin@123</code>	Single account, full platform access
Salon Owner	<code>shahrooz.alta.dev+owner0@gmail.com</code> ... <code>+owner99@gmail.com</code>	<code>Owner@123</code>	100 owners, one per seeded salon
Staff	<code>shahrooz.alta.dev+staff0@gmail.com</code> , etc.	<code>Staff@123</code>	3–8 per salon; usable via API only (no dedicated web/mobile UI)
Customer	<code>shahro.naro89+customer0@gmail.com</code> ... <code>+customer49@gmail.com</code>	<code>Customer@123</code>	50 customers, randomly assigned cities

To test the **registration + OTP flow** itself, use a fresh email not in the seed set (see 3.1) — the seeded accounts skip this step by design.

3. Web App — Role-Based Flow

Roles route to distinct sections after login, enforced by `ProtectedRoute` :

- `super_admin` / `admin` → `/admin/*`
- `salon_owner` → `/owner/*`
- `customer` → `/customer/*`

3.1 Auth Flow (all roles)

1. Go to `/register` — create an account with a new email (defaults to `customer` role).
2. Check email for OTP, then go to `/verify-otp` and submit it.
3. Go to `/login` with the new credentials.
4. **Expected:** redirect to the section matching the account's role.
5. Negative tests: wrong password, unverified account login attempt, expired/incorrect OTP, resend OTP.

3.2 Customer Flow — `/customer/*`

1. **Salons** (`/customer/salons`) — default landing page. Browse/search salon list.
2. **Salon Detail** (`/customer/salons/:id`) — view a salon's services, staff, ratings.
3. **AI Skin Scan** (`/customer/scan`) — upload/capture a face photo for AI analysis.
4. **Scan Results** (`/customer/scan-results`) — skin score, per-category breakdown (skin tone, hydration, acne, dark circles, eyebrows, lip pigmentation), treatment plan, diet plan.
5. **Matched Salons** (`/customer/matched-salons`) — salons ranked/filtered based on scan results.
6. **Booking** (`/customer/booking`) — book a service/treatment at a chosen salon.
7. **Booking History** (`/customer/bookings`) — past/upcoming bookings and statuses.
8. **Progress Report** (`/customer/progress`) — track skin improvement across repeated scans.
9. **Profile** (`/customer/profile`) — edit personal info, change password.

3.3 Salon Owner Flow — `/owner/*`

1. **Dashboard** (`/owner`) — overview metrics for the owner's salon.
2. **Services** (`/owner/services`) — add/edit/remove services (test the `aiScanLink` field tying a service to a scan concern).
3. **Events** (`/owner/events`) — manage bundled service promotions.
4. **POS** (`/owner/pos`) — point-of-sale / in-salon checkout flow.
5. **Staff** (`/owner/staff`) — manage staff members, designations, schedules.
6. **Bookings** (`/owner/bookings`) — view/manage incoming bookings.
7. **Customers** (`/owner/customers`) — customers who've booked at this salon.
8. **Reviews** (`/owner/reviews`) — view/respond to reviews.
9. **Revenue** (`/owner/revenue`) — earnings and payout summaries.
10. **Notifications** (`/owner/notifications`) — booking/system alerts.
11. **Profile** (`/owner/profile`) — edit salon owner info, toggle "South Asian specialist" flag used in scan-based matching.

3.4 Admin Flow — /admin/*

1. **Dashboard** (/admin) — platform-wide overview.
2. **Analytics** (/admin/analytics) — usage/growth metrics.
3. **Salons** (/admin/salons) — approve/reject/suspend salons (test PENDING → APPROVED transitions).
4. **Salon Owners** (/admin/owners) — **new**. List all salon owner accounts (name, phone, location, salon count, status). "+ Add Owner" creates a new owner (email/password optional — leave blank to auto-generate; generated credentials are shown once in a banner, so capture them immediately). Suspend/Activate toggles an owner's access. See Section 7 for retest steps.
5. **Bookings** (/admin/bookings) — platform-wide booking oversight, status updates.
6. **Customers** (/admin/customers) — customer account management.
7. **Reviews** (/admin/reviews) — moderate reviews.
8. **Revenue** (/admin/revenue) — platform revenue view.
9. **Payouts** (/admin/payouts) — approve/track salon payouts.
10. **Notifications** (/admin/notifications) — send/manage system notifications.
11. **Settings** (/admin/settings) — platform configuration, plus the **Admin Access** panel (**new/fixed** — previously hardcoded placeholder data). Only visible/usable to `super_admin` accounts — an `admin` -role account sees a message that only a super admin can manage admin accounts. "+ Invite Admin" creates a new `admin` -role account (optional email/password, generated credentials shown once). See Section 7.
12. **Profile** (/admin/profile) — admin account settings.

Roles note: the platform now has two admin-tier roles: `super_admin` (the original seeded account, full access including managing other admins) and `admin` (same access as `super_admin` everywhere *except* it cannot view/create/suspend other admin accounts — that's `super_admin`-only, enforced server-side).

4. Mobile App (Android) — Role-Based Flow

Install the provided APK on a device or emulator (allow install from unknown sources if sideloading). Only **customer** and **salon_owner** roles have dedicated mobile UI; admin/staff are web-only today.

4.1 Launch & Auth

1. **Splash screen** — checks for an existing session (auto token refresh) and routes to Auth or straight into the app.
2. **Auth screen** — toggle between Login / Register / Verify OTP. Same register → OTP → login sequence as web.
3. On login success, routing is role-based: `salon_owner` → Owner Dashboard; everyone else (customer) → Home.

4.2 Customer Flow (bottom nav: Home / AI Scan / Bookings / Progress)

1. **Home** — landing screen with quick actions and recommendations.

2. **AI Scan** — camera screen with live face-detection overlay, scan progress checklist (8 categories), progress animation/percentage.
3. **Scan Results** — overall score, skin tone, eyebrows, hydration, dark circles, acne, lip pigmentation, treatment plan, diet plan. Actions: "View Matched Salons," "Re-Scan."
4. **Matched Salons** — salons matched to the scan's flagged concerns.
5. **Salon Services / Salon Detail** — view a salon's offered services before booking.
6. **Booking** — supports deep entry points: plain booking, pre-selected salon, pre-selected salon + service, or AI-scan-driven booking with pre-selected treatments.
7. **Bookings List** (bottom nav "Bookings") — always reloads fresh (no cached state) — verify explicitly.
8. **Tracker / Progress** (bottom nav "Progress") — skin-progress tracking over time.
9. **Profile / Notifications** — accessible from the top app bar on any screen once logged in.

4.3 Salon Owner Flow (bottom nav: Dashboard / Calendar / Services / Insights)

1. **Owner Dashboard** — salon overview metrics.
2. **Owner Calendar** — booking calendar view.
3. **Owner Services** — manage services offered.
4. **Owner Insights** — analytics for the salon.
5. **Profile / Notifications** — same top-bar access as customer flow. Owner theme uses a distinct purple background — good visual regression check.

4.4 Session & Logout

- Logout (top bar) revokes the refresh token server-side and clears local session, returning to the Auth screen.
- Test forced logout: expire/invalidate a token server-side and confirm the app detects the 401 on the next notification poll and bounces to Auth automatically.

5. Cross-Platform Consistency Checks

Run the same test user through both clients to confirm parity:

Scenario	Web	Mobile
Register new customer → OTP → login	<code>/register</code> → <code>/verify-otp</code> → <code>/login</code>	Auth screen register → OTP mode → login
Complete AI scan, view results	<code>/customer/scan</code> → <code>/customer/scan-results</code>	AI Scan tab → Scan Results
Book via scan-recommended salon	Matched Salons → Booking	Matched Salons → Booking (AI route)
Booking appears in owner's queue	<code>/owner/bookings</code> (owner login)	Owner Calendar / Dashboard (owner login)

Booking status update reflects for customer	<code>/customer/bookings</code>	Bookings tab
Notification badge count matches unread count	Top bar bell	Top app bar bell

6. Known Gaps / Out of Scope for This Pass

- **Staff role** has no dedicated web or mobile screens yet — staff-only flows can only be exercised via the API (Postman collection) using seeded staff credentials.
- Treat any percentage-complete or "in progress" claims from other internal docs as stale unless verified against the current branch.

7. Access Management — Retest Steps

Two gaps were reported in a previous QA pass: the admin panel didn't show/manage salon owners, and super admin couldn't add/manage admin accounts through the web UI. Both are now fixed; retest as follows.

7.1 Salon Owner management

1. Log in as `super_admin` (or an `admin` -role account — both have equal access here) and go to **Admin** → **Salon Owners** (`/admin/owners`).
2. Confirm the list loads with real owner data (name, email, phone, city/country, salon count, status) — not placeholders.
3. Click **+ Add Owner**, fill in name/city/country (email/password optional), submit.
4. **Expected:** a banner shows the generated email/password if you left those blank — record these, they're needed to log in as this owner elsewhere.
5. Click **Suspend** on the new owner, confirm status flips to `suspended`. Click **Activate** to reverse it.
6. Confirm the new owner also appears in the owner dropdown when creating a salon (`/admin/salons` → + Add Salon).

7.2 Admin account management

1. Log in as the **super admin** account (`admin@hermoso.app` , or your fresh reset admin per Section 1.1) and go to **Admin** → **Settings** (`/admin/settings`).
2. In the **Admin Access** card, confirm the table shows real accounts (starting with just the super admin) — not the old hardcoded "Armaan / Sales Manager" row.
3. Click **+ Invite Admin**, fill in a name (email/password optional), submit.
4. **Expected:** a banner shows the generated email/password if left blank — record these as the fresh admin test account.
5. Log out, log in as the newly created admin account. Confirm it can reach all the same `/admin/*` pages as super admin, **including** creating salon owners (7.1).

6. Still logged in as the new admin, go to Settings → Admin Access. **Expected:** a message that only a super admin can manage admin accounts — the new admin should **not** be able to see or invite other admins, and should not be able to suspend the super admin's account.
 7. Log back in as the super admin and confirm you **can** suspend/activate the admin account created in step 3 from the Admin Access table.
-

8. Reporting Bugs

Log defects against the current branch/commit with: role used, exact test account, platform (web/mobile + OS version), steps, expected vs. actual, and screenshots. Cross-reference `docs/trello-board.md` for existing tracked issues before filing duplicates.
